Engineering and Delivery Guide

TIO OPERATIONS HUB / CONTROLLED DOCUMENT

TIO-TECH-001 | 2.0 **ISSUED | 24 OCTOBER 2026**

Control field	Controlled value
Document ID	TIO-TECH-001
Version / status	2.0 / Working technical index
Classification	Internal engineering and operations
Owner / approver	Engineering lead (proposed) / Release owner (pending)
Issue date / review	24 October 2026 / At each release
Source baseline	Repository working tree reviewed 24 October 2026; release notes identify 2.30.0
Supersedes	Initial portfolio technical pack dated 24 October 2026

1. Authority and use

This guide is a navigation and release-control aid. It deliberately points to source contracts instead of duplicating full endpoint, schema or deployment specifications. Before implementation or deployment, identify the exact commit, application version, migration set, generated SQL script, OpenAPI file, build metadata and dirty-tree state. If any artifact disagrees, stop the release review and reconcile it at source.

2. System profile

Area	Documented implementation	Verification point
API	ASP.NET Core 10 minimal API; modular monolith; versioned `/api/v1` routes	`src/TioHub.Api/Program.cs`, module registration, `docs/API.md`
Persistence	EF Core 10 and SQL Server; one shared `HubDbContext`; versioned EF migrations	`src/TioHub.Api/Data/`, migration snapshot, generated SQL
Front end	React 18, TypeScript, Vite, TanStack Query and React Router; served from API `wwwroot`	`src/web/package.json`, build and deployed asset manifest
Authentication	Windows Integrated behind IIS or Forms per approved configuration; Development auth is restricted to Development/Testing	`docs/DEPLOYMENT-GUIDE.md`, `docs/OPERATIONS.md`, configuration review
Authorization	Effective identity, roles, scopes, resource/action permissions and endpoint/workflow checks	`src/TioHub.Api/Security/`, authorization tests and negative cases
Background work	In-process scheduled services for quality, SLA evaluation, exports and related housekeeping	Configuration, logs and scale-out constraints in SDD
Files	Attachments and exports stored under configured file paths with database ownership metadata	Path ACL, backup scope, retention, export authorization tests
Reports	Server-side registry, validated filters, preview, CSV/Excel export, history, schedules and presets	`docs/DECISIONS.md` A62; current report calculation and access tests

3. Module boundaries and domain ownership

The codebase is deployed as one application, not independent services. Logical modules cover Access, Clients, Environments, Catalogue, Inventory, Network/Firewall, Work, Commitments, Contracts, Escalations, Teams, Quality, Reporting, Attachments/Notes and Administration. Cross-module data access shares the EF Core context. Avoid creating speculative message buses, repository layers or external APIs without a documented architecture decision.

Business records include customer/client and optional project context, supported technical inventory, work and event history, contract terms and reporting metadata. The database is authoritative for structured records; attachment/export bytes are file-backed. A recovery plan must coordinate database and file storage where referenced files must be preserved.

4. Interfaces and contracts

- REST route conventions, errors, paging and security expectations: API guide.
- Machine-readable API contract: OpenAPI v1. Confirm it matches the target build; do not manually patch a release copy independently from the API.
- Database deployment artifact: `database/01-schema.sql`. Review the script against all application migrations. Runtime schema mutation is not the production deployment process.
- Least-privilege identity guidance: `database/02-app-login.sql`, reviewed by the DBA/security owner before use.
- External references: Jira keys and approved links are records; the documented application does not synchronize Jira, read secret vaults, call client systems or send email.

5. Security and privacy controls to verify

At release and deployment review, verify HTTPS and IIS authentication mode; application identity and SQL grants; role/action and data-scope behavior; separation of administration from business-data access; protected operations and downloads; audit permissions and retention; setup-code handling; attachment and export paths/ACLs; secrets exclusion from package/logs; approved link and tile hosts; logging and incident access; and configured retention/backup rules. Confirm that scheduled reports execute per recipient's own access as documented.

The shared-team visibility policy does not establish customer-to-customer tenant isolation. System Design and product materials must remain aligned with the actual decision. Authorization is enforced server-side; frontend hiding alone is not evidence of access control. Tests supplement the configuration review and do not prove the security of a deployed installation.

6. Build and release sequence

The repository release source of truth is `build/Build-Release.ps1`. It builds the frontend, runs .NET tests unless explicitly skipped, publishes the API and UI, validates migration coverage against the idempotent SQL script, excludes development settings, packages deployment content, and records source and hash information. It is a local PowerShell pipeline; this repository does not provide evidence of a hosted CI service.

1. Select and record the intended commit/tag. Review working-tree status and uncommitted changes.
2. Restore locked dependencies and run the documented frontend build/type check and .NET test suite.
3. Verify all migrations are represented in `database/01-schema.sql`; review schema changes and upgrade compatibility.
4. Build the release with the approved script. Inspect package contents for development settings, secrets, unexpected files, application version, `BUILD-INFO.txt` and SHA-256.
5. Install/upgrade in an isolated environment with the target authentication, SQL permissions, storage ACLs and tile configuration.
6. Validate readiness, sign-in, authorization positive and negative paths, core workflows, reports, file handling, logs and backup/restore.
7. Obtain release, DBA, security and service-owner approvals. Apply the reviewed SQL script through the DBA process before matching application deployment. Keep rollback artifacts.

7. Schema, data and operations rules

- Never apply a migration to a shared or production database without the approved DBA change process.
- Production `Database__ApplyMigrationsOnStartup` remains false; readiness checks validate the schema and fail closed rather than migrate.
- Review additive migration assumptions, backfills, SQL generated from migrations, rollback path and compatibility with existing data.
- Back up database and required file-backed data consistently. Restore and validate in an isolated environment before switching production.
- Do not assume Git worktrees isolate SQL Server databases. Development and test configurations must use their assigned isolated databases.
- Do not include real credentials, tokens, production connection strings or customer data in documentation, screenshots, test output or packages.

8. Quality evidence and known limits

Use dated test results, not remembered test counts. Record environment, commit, commands, outcome, skipped checks and known gaps. Test evidence does not establish production authentication, performance, recoverability, compliance, external audit or customer acceptance. A production-like performance test, independent security assessment, user acceptance, capacity target, RPO/RTO and legal/privacy review require explicit owners and evidence.

The documented design is a single application instance. In-process workers, caches and file storage do not establish safe horizontal scale-out. Public map tiles may be requested by users' browsers; approve the deployment tile service and attribution. Public holidays are not modeled unless a later release implements them. Reported technical condition is user-maintained rather than live monitoring.

9. Change impact matrix

Change	Minimum companion review
API request/response behavior	OpenAPI, API guide, compatibility, tests and client release notes
Entity or index	EF migration, model snapshot, idempotent SQL, migration validation and DBA review
Role/action or scope	Security model, API checks, negative tests, admin guidance and threat review
Report definition/calculation	Registry, filter validation, access scope, definitions, export and schedule tests
Background job or retention	Configuration docs, readiness, logs, scale behavior, backup and operations runbook
UI workflow	User guide, accessibility, screenshots, browser test and training/support article
Product claim or integration	Product boundary, privacy/security, legal, commercial claim register and release notes

10. Revision history

Version	Date	Change	Approval
1.0	24 Oct 2026	Initial technical pack	Draft
2.0	24 Oct 2026	Rebuilt as source-linked engineering and release-control guide	Working draft; pending